**Министерство науки и высшего образования Российской Федерации**
Федеральное государственное бюджетное образовательное учреждение
высшего образования
**«ТОМСКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ
СИСТЕМ УПРАВЛЕНИЯ И РАДИОЭЛЕКТРОНИКИ» (ТУСУР)**

Кафедра автоматизированных систем управления (АСУ)

# МАССИВЫ И ВЕКТОРНЫЕ ОПЕРАЦИИ

Отчёт о лабораторной работе №5
по дисциплине «Вычислительная
техника»

Студент гр. 432-3
_____ Д.В. Белов
«____» _____2023 г.

Руководитель
Ассистент кафедры АСУ.
_____ А.В. Анфилофьев
«____» _____ 2023 г.

**Томск 2023**

# Оглавление

1 ЦЕЛИ И ЗАДАЧИ ПРОЕКТА.....3
1.1 Тема.....3
1.2 Цели.....3
1.3 Задание.....3
2 ХОД РАБОТЫ.....4
Вывод.....10
ПРИЛОЖЕНИЕ А Листинг программы задания.....11

# 1 ЦЕЛИ И ЗАДАЧИ ПРОЕКТА

## 1.1 Тема

Массивы и векторные операции.

## 1.2 Цели

Научиться обрабатывать массивы данных на языке Ассемблер. Познакомиться с векторными операциями процессора.

## 1.3 Задание

Задание к лабораторной работе указано на рисунке 1.1

2. ЗАДАНИЕ

Задание состоит из трёх частей. В первой части нужно выполнить задание на языке c++. Во второй части требуется написать программу обработки массива согласно заданию, используя скалярные команды обработки данных. Во второй части требуется написать такую же программу, но используя векторные операции. Для каждой программы засечь время выполнения, провести не менее 100 замеров. Вычислить среднее время выполнения для каждой программы и сделать вывод об эффективности

Рисунок 1.1 — Задание к лабораторной работе

Основное задание к лабораторной работе указано на рисунке 1.2

8. Уменьшить яркость зелёной составляющей нижней половины картинки.

Рисунок 1.2 — Основное задание к лабораторной работе

# 2 ХОД РАБОТЫ

На рисунке 2.1 Представлен код на языке C++, выполняющий задачу.

```
clock_t CPP_(unsigned char* buffer, unsigned width, unsigned height)
{
    clock_t time;
    time = clock();
    unsigned half = height / 2;
    unsigned int y;

    for (int j = half; j >= 0; j--)
    {
        y = (j * width) * 4 + 1;
        for (unsigned int i = 0; i < width; y = y + 4, i++)
        {
            buffer[y] = 10;
        }
    }
    time = clock() - time;
    return time;
}
```

Рисунок 2.1 – Код на C++

На рисунке 2.2 Представлено начало ассемблерной вставки.

```
clock_t Cx86(unsigned char* buffer, unsigned width, unsigned height)
{
    clock_t time;
    time = clock();
    _asm {
        mov edi, dword ptr[buffer]
        mov ecx, dword ptr[height] //j = ecx = height/2
        shr ecx, 1 // лог сдвиг вправо
        mov esi, dword ptr[width]
        mov bh, 0
        jmp j_firstpass

        for_j :
            dec ecx
```

Рисунок 2.2 – Начало ассемблерной вставки

На рисунке 2.3 Представлен код цикла **j_firstpass.**

```
        j_firstpass :
            mov eax, ecx
            cmp eax, 0
            jl j_end //If j<0 j_end
            imul eax, esi // j*width
            shl eax, 2 // y = j*width*4
            add eax, edi // eax = buffer + y
            add eax, 1 // eax = buffer + y + 1
            mov edx, 0 //i = edx = 0
            jmp i_firstpass
```

Рисунок 2.3 – Цикл **j_firstpass.**

На рисунке 2.4 Представлен код цикла **for_i.**

```
        for_i :
            inc edx
            add eax, 4
```

Рисунок 2.4 – Цикл **for_i**

На рисунке 2.5 Представлен код цикла **i_firstpass**.

```
i_firstpass :
    cmp edx, esi
    jae i_end //if(i>=width) -> i_end
    jnc add_light
```

Рисунок 2.5 – Цикл **i_firstpass**

На рисунке 2.6 Представлен код цикла **add_light**.

```
add_light :
    mov byte ptr[eax], 10
    jmp for_i
```

Рисунок 2.6 – Цикл **add_light**

На рисунке 2.7 Представлен код цикла **i_end.**

```
i_end :
    je for_j
    jmp for_j
```

Рисунок 2.7 – Цикл **i_end**

На рисунке 2.8 Представлен код цикла **j_end.**

```
j_end :
}
time = clock() - time;
return time;
}
```

Рисунок 2.8 – Цикл **j_end**

На рисунке 2.9 Представлен код начало на ассемблерной вставке с использованием MMX**.**

```
clock_t CmmX(unsigned char* buffer, unsigned width, unsigned height)
{
    clock_t time;
    const char* address = "%p\n";
    time = clock();
    _asm {
        mov ebx, 0x11111111
        movd mm3, ebx
        mov ebx, 0x00000000
        movd mm0, ebx
        punpcklbw mm0, mm3
        pxor mm1, mm1
        mov edi, dword ptr[buffer]
        mov ecx, dword ptr[height]
        shr ecx, 1
        mov esi, dword ptr[width]
        jmp j_firstpass
```

Рисунок 2.9 – Начало на ассемблерной вставке с использованием MMX

На рисунке 2.10 Представлен код цикла **for_j.**

```
for_j :
    dec ecx
```

Рисунок 2.10 – Цикл **for_j**

На рисунке 2.11 Представлен код цикла **j_firstpass.**

```
j_firstpass :
    mov eax, ecx
    cmp eax, 0
    jl j_end
    imul eax, esi // j*width
    shl eax, 2 // y = j*width*4
    add eax, edi // eax = buffer + y
    mov edx, 0 //i = edx = 0
    jmp i_firstpass
```

Рисунок 2.11 – Цикл **j_firstpass**

На рисунке 2.12 Представлен код цикла **for_i.**

```
for_i :
    add edx, 2
    add eax, 8
```

Рисунок 2.12 – Цикл **for_i**

На рисунке 2.13 Представлен код цикла **i_firstpass.**

```
i_firstpass :
    cmp edx, esi //i>?<width
    jae i_end //if(i>=width) -> i_end
    movq mm2, [eax]
    psubusb mm2, mm1 // вычитаем беззнаковый элемент
    movq[eax], mm2
    jmp for_i
```

Рисунок 2.13 – Цикл **i_firstpass**

На рисунке 2.14 Представлен код цикла **i_end.**

```
i_end :
    paddusb mm1, mm0 // уменьшаем яркость зеленого
    jmp for_j
```

Рисунок 2.14 – Цикл **i_end**

На рисунке 2.15  Представлен код цикла **j_end.**

```
        j_end :

        emms
    }
    time = clock() - time;
    return time;
}
```

Рисунок 2.15 – Цикл **j_end**

На рисунке 2.16  Представлен результат работы программы на С++**.**

Рисунок 2.16 – Результат программы на С++

На рисунке 2.17  Представлен результат работы программы на x86

Рисунок 2.17 – Результат программы на ассемблере

На рисунке 2.18  Представлен результат работы программы на MMX**.**

Рисунок 2.18 – Результат программы на MMX

На рисунке 2.19  Представлен результат работы 100 замеров.

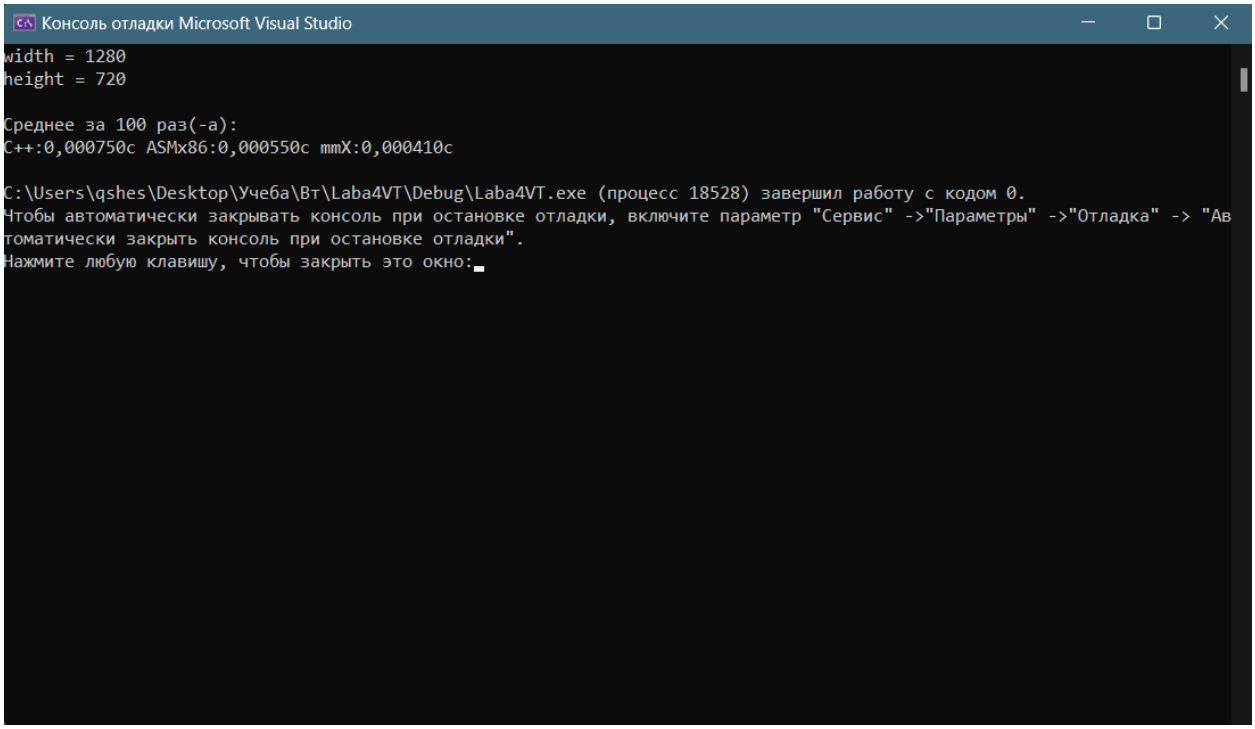


Рисунок 2.19 – Результат замеров

В приложении А представлен листинг программы.

## Вывод

Научился обрабатывать массивы данных на языке Ассемблер. Познакомиться с векторными операциями процессора.

# ПРИЛОЖЕНИЕ А
## Листинг программы задания

```
#include <iostream>
#include <ctime>

clock_t CPP_(unsigned char* buffer, unsigned width, unsigned height)
{
	clock_t time;
	time = clock();
	unsigned half = height / 2;
	unsigned int y;

	for (int j = half; j >= 0; j--)
	{
		y = (j * width) * 4 + 1;
		for (unsigned int i = 0; i < width; y = y + 4, i++)
		{
			buffer[y] = 10;
		}
	}
	time = clock() - time;
	return time;
}

clock_t Cx86(unsigned char* buffer, unsigned width, unsigned height)
{
	clock_t time;
	time = clock();
	_asm {
		mov edi, dword ptr[buffer]
		mov ecx, dword ptr[height] //j = ecx = height/2
		shr ecx, 1 // лог сдвиг вправо
		mov esi, dword ptr[width]
		mov bh, 0
		jmp j_firstpass
```

```
        for_j :
                dec ecx

        j_firstpass :
                mov eax, ecx
                cmp eax, 0
                jl j_end //If j<0 j_end
                imul eax, esi // j*width
                shl eax, 2 // y = j*width*4
                add eax, edi // eax = buffer + y
                add eax, 1 // eax = buffer + y + 1
                mov edx, 0 //i = edx = 0
                jmp i_firstpass

        for_i :
                inc edx
                add eax, 4

        i_firstpass :
                cmp edx, esi
                jae i_end //if(i>=width) -> i_end
                jnc add_light

        add_light :
                mov byte ptr[eax], 10
                jmp for_i

        i_end :
                je for_j
                jmp for_j

        j_end :
}
time = clock() - time;
return time;
```

```
}

clock_t CmmX(unsigned char* buffer, unsigned width, unsigned height)
{
	clock_t time;
	const char* address = "%p\n";
	time = clock();
	_asm {
		mov ebx, 0x11111111
		movd mm3, ebx
		mov ebx, 0x00000000
		movd mm0, ebx
		punpcklbw mm0, mm3
		pxor mm1, mm1
		mov edi, dword ptr[buffer]
		mov ecx, dword ptr[height]
		shr ecx, 1
		mov esi, dword ptr[width]
		jmp j_firstpass

		for_j :
			dec ecx

		j_firstpass :
			mov eax, ecx
			cmp eax, 0
			jl j_end
			imul eax, esi // j*width
			shl eax, 2 // y = j*width*4
			add eax, edi // eax = buffer + y
			mov edx, 0 //i = edx = 0
			jmp i_firstpass

		for_i :
			add edx, 2
			add eax, 8
```

```
            i_firstpass :
                  cmp edx, esi //i>?<width
                  jae i_end //if(i>=width) -> i_end
                  movq mm2, [eax]
                  psubusb mm2, mm1 // вычитаем беззнаковый элемент
                  movq[eax], mm2
                  jmp for_i

            i_end :
                  paddusb mm1, mm0 // уменьшаем яркость зеленого
                  jmp for_j

            j_end :

            emms
      }
      time = clock() - time;
      return time;
}

void SafeOpen(FILE*& f, const char* name, const char* mode)
{
      fopen_s(&f, name, mode);
      if (f == NULL) {
            printf("Error: Can't open %s with mode %s\n", name, mode);
            exit(-2);
      }
}

unsigned char* ReadFile(const char* name, unsigned char* header)
{
      FILE* in;
      unsigned char* buffer;
      unsigned int width, height;
      SafeOpen(in, name, "rb");
```

```
	fread(header, 1, 54, in);
	width = *(unsigned int*)(header + 18);
	height = *(unsigned int*)(header + 22);
	buffer = (unsigned char*)malloc(width * height * 4);
	fread(buffer, 1, width * height * 4, in);
	fclose(in);
	printf("width = %d\nheight = %d\n", width, height);
	return buffer;
}

void SaveFile(const char* name, unsigned char* header, unsigned char* buffer,
unsigned int len)
{
	FILE* out;
	SafeOpen(out, name, "wb");
	fwrite(header, 1, 54, out);
	fwrite(buffer, 1, len, out);
	fclose(out);
}

int main()
{
	setlocale(0, "");
	clock_t x86 = 0, mmX = 0, CPP = 0;
	double time1, time2, time3;
	int times = 100;
	unsigned char* buffer_original, * bufferCPP;
	unsigned int width, height, len;
	unsigned char headers[54];
	buffer_original = ReadFile("C:\\Users\\qshes\\Desktop\\Laba5VT\\
outNature.bmp", headers);
	width = *(unsigned int*)(headers + 18);
	height = *(unsigned int*)(headers + 22);

	len = width * height * 4;
	bufferCPP = (unsigned char*)malloc(len);
	for (int i = 0; i < times; i++) {
```

```
            memcpy(bufferCPP, buffer_original, len); //Функция memcpy копирует num
байтов первого блока памяти,
            //на который ссылается указатель srcptr, во второй блок памяти, на
который ссылается указатель destptr.
            CPP += CPP_(bufferCPP, width, height);


            memcpy(bufferCPP, buffer_original, len);
            x86 += Cx86(bufferCPP, width, height);


            memcpy(bufferCPP, buffer_original, len);
            mmX += CmmX(bufferCPP, width, height);
        }
        time1 = ((double)CPP / CLOCKS_PER_SEC) / times;
        time2 = ((double)x86 / CLOCKS_PER_SEC) / times;
        time3 = ((double)mmX / CLOCKS_PER_SEC) / times;


        printf("\nСреднее за %d раз(-а):\nC++:%lfc ASMx86:%lfc mmX:%lfc\n", times,
time1, time2, time3);


        memcpy(bufferCPP, buffer_original, len);
        CmmX(bufferCPP, width, height);
        SaveFile("C:\\Users\\qshes\\Desktop\\Laba5VT\\outCmmX.bmp", headers,
bufferCPP, len);


        memcpy(bufferCPP, buffer_original, len);
        CPP_(bufferCPP, width, height);
        SaveFile("C:\\Users\\qshes\\Desktop\\Laba5VT\\/outCPP.bmp", headers,
bufferCPP, len);


        memcpy(bufferCPP, buffer_original, len);
        Cx86(bufferCPP, width, height);
        SaveFile("C:\\Users\\qshes\\Desktop\\Laba5VT\\outCx86.bmp", headers,
bufferCPP, len);


        free(bufferCPP);
        free(buffer_original);
        return 0;
}
```